

Bug 2,

Test problem

- BlockchainTest

- Does NOT test:

- fromAddress or toAddress being null or "".
 - Negative fees.
 - Invalid block numbers (<0 or >= size) in getBlockByNumber() .

- BlockTest

- No tests with null transactions or zero/negative amounts.

- GenericPQTest

- No tests for inserting null elements.

- ListoverLinkedListTest

- No tests with null elements.

- AVLTreeTest

- No tests with null keys or values.

Fix exception!!!

AVLTreeImp

```
java
1  @Override
2  public void insertTree(K key, V value) {
3      //Add new exception!!!
4      if (key == null) {
5          throw new IllegalArgumentException("AVLTree.insertTree(): key
6 must not be null.");
7      }
8      //finish
9      AVLNode<K, V> node = searchNode(root, key);
10     if ( node != null ) {
11         throw new IllegalArgumentException(
12             "AVLTree.insertTree(): key already exists in AVL tree."
13         );
14     }
15     this.root = insertNode(this.root, null, key, value);
```

```

16         nodesNum++;
17         print();
18         if ( !repOK() ) {
19             throw new IllegalStateException(
20                 "AVLTree.insertTree(): representation invariant of AVL tree
21 is not satisfied."
22             );
23         }
24     }
25
26     @Override
27     public V searchTree(K key) {
28         //Add new exception!!!
29         if (key == null) {
30             throw new IllegalArgumentException("AVLTree.searchTree(): key
31 must not be null.");
32         }
33         //finish
34         AVLNode<K, V> keyNode = searchNode(this.root, key);
35         if (keyNode != null) return keyNode.val;
36         else return null;
37     }
38
39     @Override
40     public void updateTree(K key, V newValue) {
41         //Add new exception!!!
42         if (key == null) {
43             throw new IllegalArgumentException("AVLTree.updateTree(): key
44 must not be null.");
45         }
46         //finish
47         AVLNode<K, V> keyNode = searchNode(this.root, key);
48         if ( keyNode == null ) throw new NoSuchElementException(
49             "AVLTree.updateTree(): key does not exist in AVL tree."
50         );
51         else keyNode.val = newValue;
52     }

```

BalanceImp.java

```

java
1     @Override
2     public int getBalance(String address) {
3         //add new exception!!!!
4         if (address == null || address.isEmpty()) {

```

```

5         throw new IllegalArgumentException("address cannot be null or
6 empty");
7     }
8     //finish
9     int balanceOfAddress;
10    AddressBalancePair targetPair = addBalPairs.searchTree(address); //
11    O(log |A|)
12    if ( targetPair != null ) balanceOfAddress = targetPair.balance;
13    else balanceOfAddress = 0;
14    return balanceOfAddress;
15 }

```

and

```

java
1    @Override
2    public void updateBalance(String address, int newBalance) {
3        //add new exception!!!!
4        if (address == null || address.isEmpty()) {
5            throw new IllegalArgumentException("address cannot be null or
6 empty");
7        }
8        if (newBalance < 0) {
9            throw new IllegalArgumentException("address cannot be null or
10 empty");
11        }
12        //finish
13        AddressBalancePair targetPair = addBalPairs.searchTree(address);
14    // O(log |A|)
15        int oldBalance = (targetPair == null) ? 0 :
16 targetPair.balance; //fix totalSupply doesn't update
17        if ( address.equals("0") && allAddress.length() == 0 ) {
18            // initiat `totalSupply` with the amount of BB-coins of
19            // the address "0" in the genesis block
20            //bug!!! addBalPairs.insertTree(address, targetPair);
21            addBalPairs.insertTree(address, new AddressBalancePair(address,
22 newBalance));
23            allAddress.insertRear(address);
24    // O(log |A|)
25            //fix using above 2 line
26            totalSupply = newBalance;
27        } else if ( targetPair == null ) {
28            // address does not exist in Balance
29            addBalPairs.insertTree(address, new AddressBalancePair(address,
30 newBalance)); // O(log |A|)

```

```

31         allAddress.insertRear(address);
32 // 0(1)
33         totalSupply += newBalance;
34     } else {
35         // address already exists in Balance
36         addBalPairs.updateTree(address, new AddressBalancePair(address,
newBalance));
        totalSupply += (newBalance - oldBalance);
    }
    if ( !repOK() ) {
        throw new IllegalStateException(
            "BalanceImp.updateBalance(): totalSupply does not equal to
the sum of all accounts."
        );
    }
}

```